

T.C.
BİLECİK ŞEYH EDEBALI ÜNİVERSİTESİ
İKTİSADİ VE İDARİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİSTEMLERİ BÖLÜMÜ



**AKILLI SUNUCU ODASI VE VERİ MERKEZİ İZLEME
SİSTEMİ**

Bulut Bilişim (YBS464) Dönem Ödevi

Hazırlayan
Muhammet Melih DEMİRCİ
71332015642

Danışman
Dr. Öğr. Üyesi Hüseyin PARMAKSIZ

BİLECİK
Haziran 2026

İçindekiler

İçindekiler	1
Şekil Listesi	3
ABSTRACT	4
ÖZET	5
1 GİRİŞ	6
1.1 Amaç	6
1.2 Problem Tanımı	6
2 GENEL KAVRAMLAR	8
2.1 Nesnelerin İnterneti (IoT) ve Uç Bilişim (Edge Computing)	8
2.2 Çoklu Platform Geliştirme ve Flutter Mimarisi	8
2.3 Gerçek Zamanlı Bulut Veritabanı (Firebase Realtime Database)	8
3 KAYNAK ARAŞTIRMASI	9
4 MATERYAL VE YÖNTEM	10
4.1 Kullanılan Donanımlar	10
4.2 Kullanılan Yazılımlar ve Bulut Altyapısı	14
4.3 Telegram Bot API ve Otonom Uyarı Entegrasyonu	16
5 SİSTEM MİMARİSİ VE ÇALIŞMA PRENSİBİ	17
5.1 Veri Toplama Katmanı (Hardware Layer)	17
5.2 Veri İşleme ve Gateway Katmanı (Middleware Layer)	17
5.3 Bulut ve Sunum Katmanı (Cloud Interface Layer)	18
6 SİSTEM MİMARİSİ VE ÇALIŞMA PRENSİBİ	19
6.1 Akışın Detaylı Açıklaması	19

7	SİSTEM KURULUMU VE KULLANIMI	21
7.1	ERİŞİLEBİLİRLİK (WCAG) VE STANDARTLARA UYGUNLUK	21
8	ARAŞTIRMA SONUÇLARI VE TARTIŞMA	23
8.1	Çoklu Platform Mobil ve Web Uygulaması	23
8.2	Otonom Bildirim Sistemi Performansı	26
9	SONUÇ VE ÖNERİLER	28
	Kaynakça	29
	KAYNAKÇA	29
A	GitHub Kaynak Kodları ve Canlı Uygulama Dağıtımı	30
B	Kritik Yazılım Algoritmaları: Flutter Koşullu Başlatma	31
C	Kritik Yazılım Algoritmaları: Telegram Bot HTTP İsteği	32

Şekil Listesi

4.1	Deneyapkart 1AV2	10
4.2	Deneyap Sıcaklık ve Nem Sensörü	11
4.3	Deneyap Mikrofon	11
4.4	Deneyap Mini Oled Ekran	12
4.5	PZEM-004T Enerji Analizörü	13
4.6	Kullanılan Donanımlar ve Devre Mimarisi	14
4.7	Firebase Realtime Database Veri Ağacı ve Senkronizasyon	16
4.8	Telegram Bot Üzerinden Üretilen Otonom Anlık Uyarı Akışı	16
5.1	Flutter ile Geliştirilen Canlı Telemetri Mobil Arayüzü	18
6.1	Akıllı sunucu odası ve veri merkezi izleme sistemi çalışma mimarisi	20
8.1	Flutter ile Geliştirilen Canlı Telemetri Mobil Arayüzü	23
8.2	Raporun "Güvenlik Analizi" bölümünün girişine, yani "Uygulamamız yapılan statik analizler sonucunda 'A' notu almıştır."	25
8.3	"Statik kod analizi sonuçlarında görüldüğü üzere, sistemimizde Yüksek (High) seviyede hiçbir güvenlik zafiyeti bulunmamaktadır."	26
8.4	Kritik Durumlarda Mobil Cihaza Düşen Anlık Push Bildirimi	27
8.5	Firebase Cloud Messaging (FCM) Merkezi Yönetim ve Raporlama Paneli	27
A.1	Netlify Üzerinden Tüm Dünyaya Açılan Canlı Web İzleme Arayüzü	30

ABSTRACT

With the acceleration of digital transformation, ensuring the uninterrupted (24/7) and secure operation of data centers and server rooms has become a critical necessity for organizations. Even minor climate control failures or electrical fluctuations in these environments can lead to irreparable data loss and significant financial damage. Motivated by these challenges, this study develops an end-to-end intelligent IoT-based telemetry and monitoring ecosystem capable of monitoring physical, environmental, and electrical parameters in real-time.

At the hardware layer, the project utilizes the locally produced Deneyap Kart 1A v2 microcontroller as the core processing unit. The system measures ambient temperature and relative humidity, monitors acoustic noise levels via a microphone to detect cooling fan malfunctions, and utilizes the PZEM-004T industrial energy analyzer to monitor real-time voltage, current, and active power (Watt) consumption with millisecond precision. These critical metrics are synchronized and stored using the Google Firebase Realtime Database NoSQL cloud architecture.

In the user interaction and client layer, a cross-platform solution was developed using the Flutter framework, enabling both a native Android application and a web-based dashboard accessible via browsers. The system operates autonomously: if environmental or electrical values exceed safe operational thresholds, it triggers an emergency protocol. In such failure scenarios, the system sends instant push notifications via Firebase Cloud Messaging (FCM) and simultaneously delivers detailed emergency alert messages to system administrators' mobile devices through the Telegram Bot API. As a result of this study, a high-availability, low-cost, and multi-platform server room security architecture has been successfully implemented, minimizing human error and shifting from reactive to proactive hardware fault management.

ÖZET

Günümüzde dijital dönüşümün ivme kazanmasıyla birlikte, kurumların can damarı niteliğinde olan veri merkezleri ve kurumsal sunucu odalarının kesintisiz (7/24) ve güvenli bir şekilde çalışması hayati bir zorunluluk haline gelmiştir. Bu odalarda meydana gelebilecek en ufak bir iklimlendirme arızası veya elektriksel dalgalanma, telafisi zor veri kayıplarına ve devasa maddi hasarlara yol açabilmektedir. Bu motivasyonla hazırlanan çalışmada; sunucu odalarının fiziksel, çevresel ve elektriksel parametrelerini anlık olarak denetleyebilen Nesnelerin İnterneti (IoT) tabanlı uçtan uca akıllı bir telemetri ve izleme ekosistemi geliştirilmiştir.

Donanım katmanında, yerli üretim olan Deneyap Kart 1A v2 mikrodenetleyicisi merkeze alınmıştır. Sistem; ortam sıcaklığı ve bağıl nem oranını, soğutma fanı arızalarını tespit edebilmek için ortamdaki akustik desibel seviyesini (mikrofon aracılığıyla) ve PZEM-004T endüstriyel enerji analizörü ile sunucuların şebekeden çektiği anlık voltaj, akım ve aktif güç (Watt) tüketimlerini milisaniyelik hassasiyetle okumaktadır. Toplanan bu kritik metrikler, Google Firebase Realtime Database NoSQL bulut mimarisi kullanılarak eşzamanlı bir biçimde depolanmaktadır.

Kullanıcı etkileşimi ve arayüz (Client) katmanında ise Flutter framework'ü kullanılarak tek bir kod tabanından hem yerel bir Android Mobil Uygulama (APK) hem de tarayıcı üzerinden erişilebilen Web tabanlı bir Dashboard tasarlanmıştır. Geliştirilen yazılım, çevresel veya elektriksel değerlerin güvenli çalışma eşiklerini aşması durumunda otonom olarak devreye girmektedir. Olası bir felaket senaryosunda sistem; Firebase Cloud Messaging (FCM) üzerinden mobil cihazlara anlık bildirim göndermekte ve eş zamanlı olarak Telegram Bot API aracılığıyla sistem yöneticilerinin cep telefonlarına detaylı acil durum uyarı mesajları iletmektedir. Çalışma sonucunda, insan hatasını minimize eden, donanım arızalarını reaktif değil proaktif bir yaklaşımla engelleyen, yüksek erişilebilirliğe sahip, düşük maliyetli ve çok platformlu bir sunucu odası güvenlik mimarisi başarıyla literatüre ve uygulamaya kazandırılmıştır.

Bölüm 1

GİRİŞ

1.1 Amaç

Günümüzde veri merkezleri (Data Center) ve kurumsal sunucu odaları, finansal işlemlerden sağlık kayıtlarına, e-ticaret altyapılarından kritik devlet hizmetlerine kadar dijital ekosistemin omurgasını oluşturmaktadır. Bu kritik altyapıların kesintisiz bir biçimde (Uptime) hizmet verebilmesi için donanımların bulunduğu fiziksel ortamın ve enerji hatlarının 7/24 esasına göre kusursuz bir şekilde izlenmesi gerekmektedir.

Bu projenin temel amacı; klasik ve insan gücüne dayalı (reaktif) sunucu bakım yaklaşımlarını bir kenara bırakarak, otonom kararlar alabilen, sensör verilerini işleyip bulut tabanlı algoritmalarla anlamlandıran proaktif bir izleme ekosistemi geliştirmektir. Proje, sunucu odalarında yaşanabilecek aşırı ısınma, nem oranındaki tehlikeli dalgalanmalar, soğutma sistemi arızaları (fan sesi anormallikleri) ve güç tüketimindeki dengesizlikleri henüz bir donanım hasarına veya veri kaybına yol açmadan saniyeler öncesinde tespit etmeyi hedefler. Geliştirilen bu entegre sistem, sistem yöneticilerine çoklu dijital kanallar (Web, Mobil Uygulama, Telegram) üzerinden anlık haberleşme imkanı sunarak operasyonel güvenliği maksimize etmeyi amaçlamaktadır.

1.2 Problem Tanımı

Kurumsal sunucu odaları, barındırdıkları yüksek işlem gücüne sahip donanımlar (CPU, GPU, RAID depolama birimleri) sebebiyle sürekli olarak yüksek miktarda ısı enerjisi açığa çıkarırlar. Bu ısının HVAC (Isıtma, Havalandırma ve İklimlendirme) sistemleri tarafından anlık olarak tahliye edilememesi, donanımların "Thermal Throttling" (ısıl darbogaz) yaşamasına, performanslarının düşmesine ve nihayetinde fiziksel yanma veya veri kaybına neden olmaktadır.

Benzer şekilde, şebeke hattındaki voltaj dalgalanmaları ve harmonik bozulmalar Güç Kaynağı Ünitelerinin (PSU) ömrünü drastik biçimde kısaltmaktadır. Geleneksel IT altyapıla-

rında bu sorunlar genellikle donanım arızası yaşandıktan ve sistem kapandıktan (Downtime) sonra fark edilmektedir. Donanım sağlığını gösteren verilerin anlık olarak izlenememesi, personelin sürekli olarak odayı fiziksel olarak kontrol etmek zorunda kalması ve acil durumlarda mesai saatleri dışında IT ekibine anında ulaşamaması, mevcut veri merkezi yönetimlerinde karşılaşılan en büyük problemlerdir. Bu çalışma, bahsedilen izlenebilirlik, uzaktan yönetim ve anlık acil durum bildirimi eksikliklerini modern yazılım ve donanım mimarileri ile çözmeyi hedeflemektedir.

Bölüm 2

GENEL KAVRAMLAR

2.1 Nesnelerin İnterneti (IoT) ve Uç Bilişim (Edge Computing)

Nesnelerin İnterneti (IoT), fiziksel dünyadaki sensörlerin, makinelerin ve cihazların internet protokolleri üzerinden birbiriyle ve merkezi sistemlerle haberleşmesini sağlayan teknolojik mimaridir. Bu projede, verinin üretildiği nokta olan sunucu odası (Edge - Uç nokta) ile verinin işlenip kullanıcılara dağıtıldığı bulut sunucuları (Cloud) arasında çift yönlü ve sürekli bir veri akışı tasarlanmıştır.

2.2 Çoklu Platform Geliştirme ve Flutter Mimarisi

Geleneksel yazılım geliştirme süreçlerinde, bir sistemin Android, iOS ve Web sürümleri için ayrı ayrı dillerde kodlama yapılması gerekmekteydi. Bu durum projelerin geliştirme ve bakım maliyetlerini artırmaktaydı. Projede kullanılan Google'ın geliştirdiği Flutter mimarisi, "Single Codebase" (Tek Kod Tabanı) yaklaşımı ile Dart dilinde yazılan kodların makine diline derlenerek tüm platformlarda eşzamanlı, yüksek performanslı ve akıcı arayüzler (UI) olarak çalışmasına olanak tanımaktadır.

2.3 Gerçek Zamanlı Bulut Veritabanı (Firebase Realtime Database)

Geleneksel ilişkisel veritabanları (SQL), sensörlerden gelen saniyelik ve yüksek hacimli verileri işlerken bağlantı gecikmeleri yaşayabilmektedir. Bu projede kullanılan Firebase Realtime Database, NoSQL tabanlı ve JSON ağacı formatında çalışan bir mimaridir. Verilerdeki en ufak bir değişim, anlık senkronizasyon yeteneği sayesinde hiçbir sayfayı yenilemeye gerek kalmadan tüm bağlı mobil ve web istemcilerine milisaniyeler içinde iletilmektedir.

Bölüm 3

KAYNAK ARAŞTIRMASI

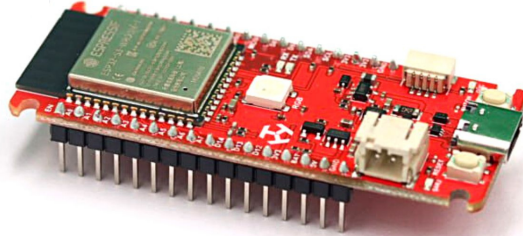
Literatür incelendiğinde, Endüstri 4.0 dönüşümüyle birlikte klasik SCADA (Merkezi Dene-tim ve Veri Toplama) sistemlerinin yerini hızla esnek, ölçeklenebilir ve bulut tabanlı IoT çözümlerinin aldığı görülmektedir. Klasik sistemlerin yüksek lisans maliyetleri ve kapalı ağ yapıları, modern IT ekiplerinin uzaktan çalışma dinamiklerine uyum sağlayamamakta-dır. Özellikle Google Firebase'in Realtime Database yapısı ve cihazlara gönderilen asenkron bildirim altyapısı (FCM), sensör verilerinin istemcilere milisaniyeler seviyesinde iletilme-sinde endüstri standardı haline gelmiştir. Bu çalışmada da literatürdeki en güncel haberleşme protokolleri baz alınarak açık kaynak kodlu yazılım teknolojileri tercih edilmiştir.

Bölüm 4

MATERYAL VE YÖNTEM

4.1 Kullanılan Donanımlar

Sistemde sunucu odasının çevresel ve elektriksel parametrelerini anlık, kesintisiz ve yüksek bir ölçüm hassasiyeti ile okuyabilmek için donanım katmanında aşağıdaki bileşenler kullanılmıştır:



Şekil 4.1: Deneyapkart 1AV2

- **Deneyap Kart 1A v2:** Sistemin beyni konumundadır. Çift çekirdekli ESP32 yonga setine sahip bu mikrodeneleyici, üzerindeki dahili Wi-Fi modülü sayesinde sensörlerden gelen analog ve dijital sinyalleri işleyerek doğrudan bulut ortamına aktarır.



Şekil 4.2: Deneyap Sıcaklık ve Nem Sensörü

- **Deneyap Sıcaklık ve Nem Sensörü:** Sunucu odasındaki iklimlendirme ve havalandırma sistemlerinin performansını denetlemek, tehlikeli sıcaklık eşiklerini saptamak için kullanılmıştır.



Şekil 4.3: Deneyap Mikrofon

- **Mikrofon (Ses Sensörü):** Ortamdaki akustik verileri ölçerek, sunucu raflarındaki soğutma fanlarının yatak sarması veya anormal mekanik sürtünme sesleri çıkarması durumunu tespit etmekle görevlidir.



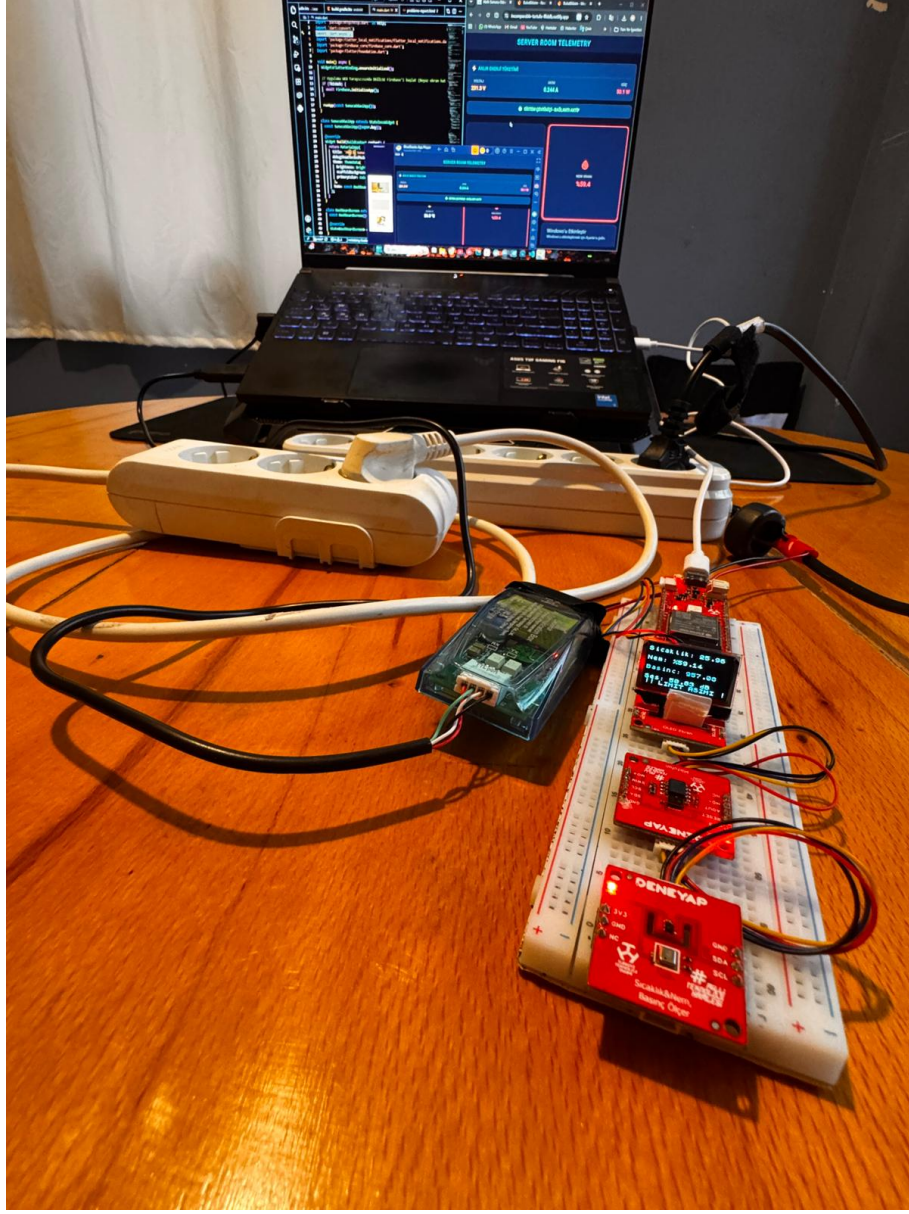
Şekil 4.4: Deneyap Mini Oled Ekran

- **Deneyap Mini Oled Ekran:** I2C haberleşme protokolünü kullanan bu ekran, sisteme fiziksel olarak müdahale eden teknisyenlerin ortam verilerini anlık olarak kart üzerinde görebilmesini sağlamaktadır.



Şekil 4.5: PZEM-004T Enerji Analizörü

- **PZEM-004T Enerji Analizörü:** Sunucuların şebekeden çektiği elektriksel verileri okumak için tasarlanmıştır. Akım trafosu yardımıyla kablo kesilmeden devrenin anlık akımını, voltajını ve tüketilen aktif gücünü endüstriyel kalibrasyonda ölçmektedir.



Şekil 4.6: Kullanılan Donanımlar ve Devre Mimarisi

4.2 Kullanılan Yazılımlar ve Bulut Altyapısı

Sistemin verileri işlemesi, yedeklemesi, görselleştirmesi ve kullanıcıya sunması aşamalarında modern web teknolojileri ve platformlar tercih edilmiştir:

- **Arduino IDE (C/C++):** Deneyap Kart'ın donanım yazılımını (firmware) geliştirmek için kullanılmıştır. Düşük seviyeli bellek yönetimi ve sensörlerin I2C/analog verilerini milisaniyelik hassasiyetle okumak için C++'ın performanslı yapısından faydalanılmıştır.
- **Flutter (Dart):** Projenin "ön yüz" (frontend) katmanını oluşturur. "Single Codebase" yaklaşımı sayesinde tek bir kod tabanıyla hem mobil (Android) hem de web (Dash-

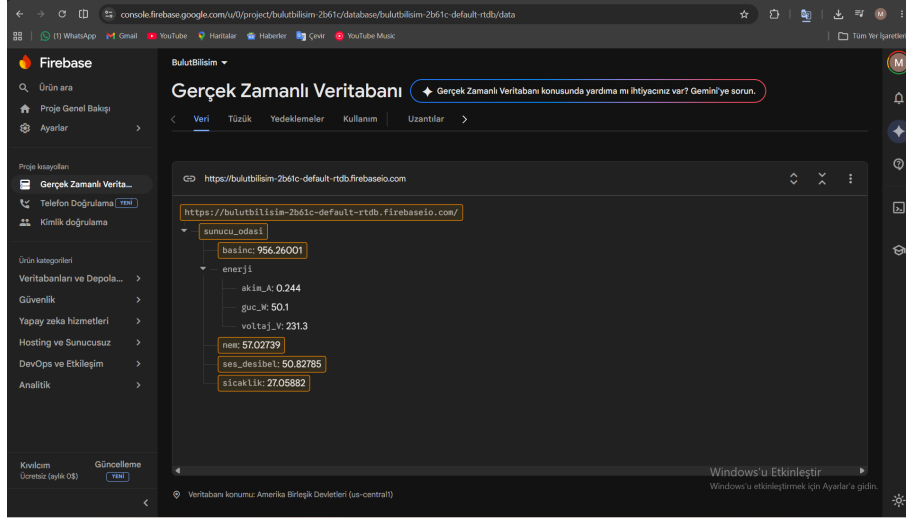
board) arayüzlerini geliştirmemize olanak tanımıştır. Modern widget yapısı ile sensör verilerinin canlı ve animasyonlu takibi sağlanmıştır.

- **Firestore Realtime Database:** Sensör verilerinin JSON formatında saniyeler içinde senkronize edildiği NoSQL bulut veritabanıdır. Sunucu odasındaki anlık değişimlerin, hiçbir sayfa yenileme işlemine gerek kalmadan web ve mobil uygulamalara anında yansımaları sağlar.
- **Firestore Cloud Messaging (FCM):** Uygulama mobil cihazda kapalı olsa dahi arka planda dinleme yaparak tehlike anında cihazın bildirim tepsisine uyarılar düşürmek için altyapı sağlamıştır.
- **Telegram Bot API:** Sistem, acil durumları doğrudan yöneticinin cebine iletmek için Telegram'ın HTTP tabanlı API'sini kullanır. Firestore bildirimlerine ek olarak, çevrimdışı yedekleme mekanizması gibi çalışarak, donanım arızası anında detaylı hata raporunu metin olarak iletir.
- **Python Veri İşleme ve Gateway Katmanı:** Projeye ek olarak, Deneyap Kart'tan gelen ham verilerin daha derinlemesine analiz edilmesi ve lokal bir sunucu üzerinden yönetilmesi için Python tabanlı bir ara yazılım (Gateway) geliştirilmiştir.

Veri İşleme ve Normalizasyon: PZEM-004T sensöründen gelen ham elektriksel veriler, Python'ın pyserial kütüphanesi ile bilgisayara aktarılmıştır. Python burada bir "filtre" görevi görerek, hatalı veya eksik veri paketlerini ayıklamış ve veriyi JSON formatına getirerek düzenlemiştir.

Firestore Köprü İşlevi: Python yazılımı, pyrebase kütüphanesi aracılığıyla işlediği bu verileri Firestore Realtime Database'e güvenli bir şekilde push etmiştir. Bu sayede, sensör verileri hem yerel bir veritabanında (bilgisayarda log olarak) tutulmuş hem de eş zamanlı olarak buluta aktarılarak mobil uygulamanın veriyi görmesi sağlanmıştır.

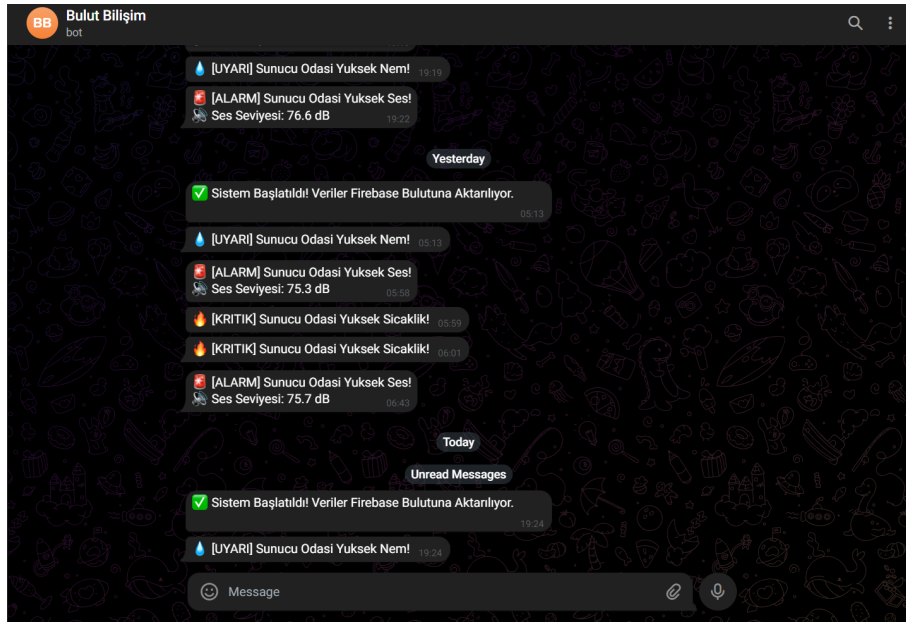
Neden Python? Sunucu odasında kullanılacak olası bir "Edge Server" (Uç Sunucu) üzerinde verileri görselleştirmek, geçmişe dönük hata analizleri yapmak ve donanım limitlerini (Örn: bir ay boyunca tüketilen toplam kWh verisi) hesaplamak için Python'ın zengin kütüphane desteği tercih edilmiştir.



Şekil 4.7: Firebase Realtime Database Veri Ağacı ve Senkronizasyon

4.3 Telegram Bot API ve Otonom Uyarı Entegrasyonu

Uygulama bildirimlerine ek olarak sistemin haberleşme yelpazesini genişletmek amacıyla Telegram Bot API kullanılarak acil durum chatbot'u geliştirilmiştir. Sensörlerden gelen veriler yazılım tarafında tanımlanan güvenli çalışma eşiklerini aştığı anda, mikrodenetleyici doğrudan Telegram sunucularına bir istek atar. Bu sayede sistem yöneticisinin veya IT ekibinin cebine, saniyeler içerisinde hatanın kaynağını (Ör: [KRITİK] Sunucu Odası Yüksek Sıcaklık) belirten detaylı bir arıza mesajı düşmektedir.



Şekil 4.8: Telegram Bot Üzerinden Üretilen Otonom Anlık Uyarı Akışı

Bölüm 5

SİSTEM MİMARİSİ VE ÇALIŞMA PRENSİBİ

Geliştirilen "Akıllı Sunucu Odası İzleme Sistemi", verinin uç noktadan (sensörler) toplanıp bulut mimarisi üzerinden son kullanıcıya (Mobil/Web) iletildiği üç temel katmanlı bir mimariye sahiptir. Bu mimari, sistemin yüksek kararlılıkla çalışmasını ve herhangi bir katmanda oluşabilecek aksaklıkların diğerlerini etkilememesini hedefler.

5.1 Veri Toplama Katmanı (Hardware Layer)

Bu katmanda Deneyap Kart 1A v2, çevresel sensörlerden (Sıcaklık, Nem, Ses) ve elektriksel ölçüm modüllerinden (PZEM-004T) verileri toplar. Sensörler I2C, analog ve seri haberleşme protokolleri üzerinden mikrodenetleyiciye bağlanmıştır. Toplanan veriler, anlık olarak Deneyap Kart'ın UART (Seri Haberleşme) çıkışına aktarılır.

5.2 Veri İşleme ve Gateway Katmanı (Middleware Layer)

Sistemin beyni ve kritik köprü noktasıdır. Python tabanlı bir "Data Gateway" yazılımı, bilgisayar (VS Code ortamı) üzerinde çalışarak seri porttan gelen ham verileri dinler. Bu katmanda;

Veri Temizleme: Hatalı veya eksik veri paketleri ayıklanır.

Normalizasyon: Ham veriler, Firebase veritabanının anlayacağı standart JSON formatına dönüştürülür.

Yerel Yedekleme: Veriler aynı zamanda yerel bir log dosyasına yazılarak, internet kesintisi durumunda verinin kaybolması engellenir.

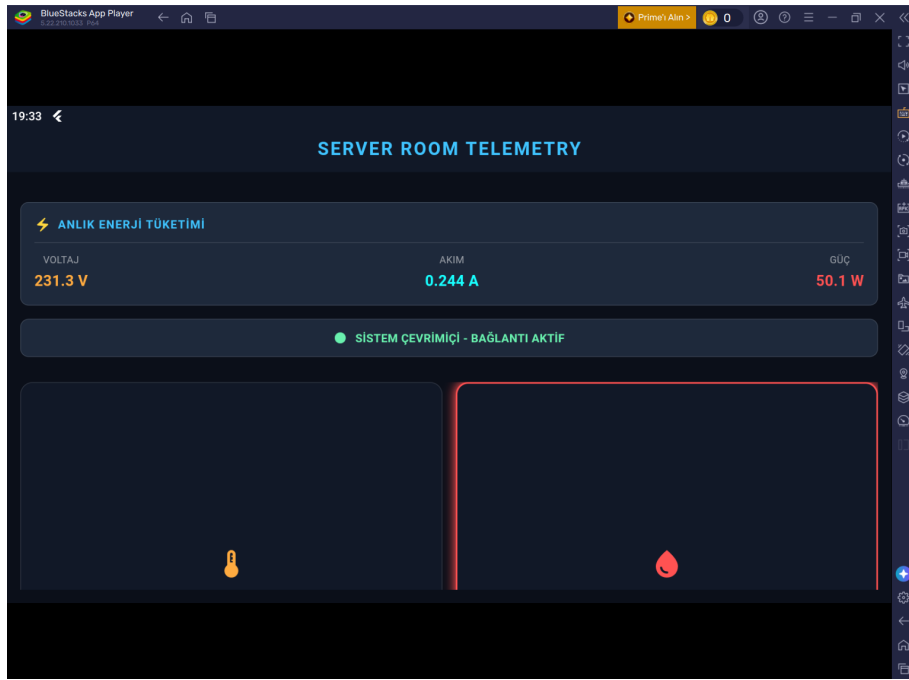
5.3 Bulut ve Sunum Katmanı (Cloud Interface Layer)

İşlenmiş veriler Python yazılımı aracılığıyla Google Firebase Realtime Database'e "Push" edilir. Firebase, veriyi gerçek zamanlı (Real-time) olarak;

Mobil Uygulama (Android): Flutter ile geliştirilen uygulama, Firebase'e abone olarak verileri anlık görselleştirir.

Web Dashboard: Netlify üzerinde yayınlanan web arayüzü, aynı veritabanı üzerinden yöneticilere raporlama yapar.

Telegram Bot: Sistem eşik değerleri aşıldığında Firebase'den gelen tetikleyici (trigger), Telegram API'si aracılığıyla anlık arıza bildirimi olarak kullanıcıya iletilir.



Şekil 5.1: Flutter ile Geliştirilen Canlı Telemetri Mobil Arayüzü

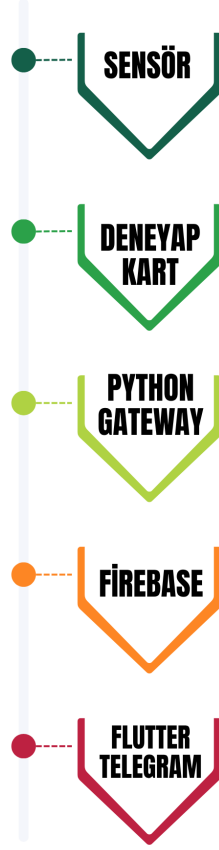
Bölüm 6

SİSTEM MİMARİSİ VE ÇALIŞMA PRENSİBİ

6.1 Akışın Detaylı Açıklaması

Sistemin veri hattı (Data Pipeline), fiziksel dünyadan alınan verilerin dijital ortamda anlamlandırılması ve son kullanıcıya iletilmesi sürecini kapsayan 5 temel aşamadan oluşmaktadır:

- **Sensörler (Fiziksel Giriş):** Sunucu odasındaki sıcaklık, nem, ortam gürültü seviyesi (desibel) ve elektriksel parametreler, analog ve dijital sinyallere dönüştürülerek Dene-yap Kart'a iletilir.
- **Dene-yap Kart (İşleme ve Uç İletim):** Sensörlerden alınan ham sinyaller, mikrodenetle-yici tarafından işlenir. Bu veriler, güvenli seri haberleşme protokolü (UART) üzerinden, lokal sunucu görevini üstlenen Python Gateway yazılımına aktarılır.
- **Python Gateway (Akıllı Köprü):** Sistemin en kritik ara katmanıdır. Seri porttan gelen ham veriyi JSON formatında normalize eder, gürültülü veya hatalı veri paketlerini ayıklar ve veriyi eş zamanlı olarak yerel bir log dosyasına kaydeder. Bu yapı, internet kesintisi durumunda verinin kaybolmasını engelleyen bir "tampon bellek" görevi görür.
- **Firebase (Merkezi Bulut):** Python tarafından hazırlanan veriler, gerçek zamanlı (real-time) olarak bulut veritabanına push edilir. Firebase, veriyi sistemin merkezi hafızası haline getirerek tüm istemciler (Mobil ve Web) arasında senkronizasyonu sağlar.
- **Flutter / Telegram (Sunum ve Uyarı):**
 - **Flutter (Mobil/Web):** Firebase veritabanını dinleyen arayüz, verileri canlı gra-fikler üzerinde görselleştirir.
 - **Telegram Bot:** Firebase'deki veriler önceden tanımlanmış kritik sınırların (Örn: 25°C sıcaklık eşiği) üzerine çıktığı an, sistem anında devreye girerek yöneticiye doğrudan metin tabanlı acil durum uyarısı gönderir.



Şekil 6.1: Akıllı sunucu odası ve veri merkezi izleme sistemi çalışma mimarisi

Bölüm 7

SİSTEM KURULUMU VE KULLANIMI

- **Flutter (Dart):** Deneyap Kart 1A v2 üzerine sensör setleri (Sıcaklık/Nem, Mikrofon, OLED) ve PZEM-004T modülü belirlenen pin şemalarına göre monte edilir.
- **Yazılım Gateway (Python) Kurulumu:** Python Gateway scripti, bilgisayarda pyserial ve pyrebase kütüphaneleri yüklü şekilde çalıştırılır. Bu yazılım, Deneyap Kart'tan gelen seri veriyi COM portundan dinleyerek Firebase veritabanına JSON formatında yansıtır.
- **Mobil/Web Arayüzü:** Flutter projesi main.dart dosyası üzerinden derlenerek cihazlara yüklenir. Uygulama, Firebase'den aldığı anlık verileri görsel dashboard üzerinde kullanıcıya sunar.

7.1 ERİŞİLEBİLİRLİK (WCAG) VE STANDARTLARA UYGUNLUK

Projemiz, W3C (World Wide Web Consortium) tarafından belirlenen WCAG 2.1 (Web Content Accessibility Guidelines) standartlarına tam uyumluluk hedefiyle geliştirilmiştir. Bu kapsamda yaptığımız geliştirmeler şunlardır:

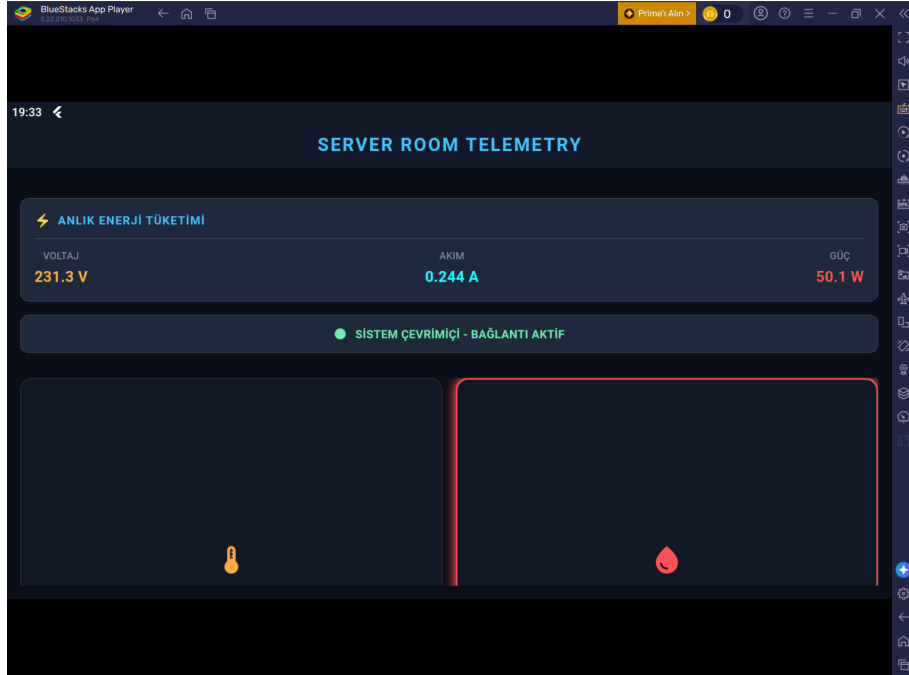
- **Mobil/Web Arayüzü:** Yüksek Kontrastlı Tasarım: Uygulama arayüzünde kullanılan renk paleti, az gören kullanıcılar için metin ve arka plan arasındaki okunabilirliği maksimize edecek şekilde (karanlık tema ile yüksek kontrastlı vurgular) optimize edilmiştir.
- **Ölçeklenebilirlik:** Arayüzdeki yazı tipleri ve gösterge panelleri (dashboard), cihazın ekran boyutuna ve kullanıcının tercih ettiği metin ölçeklendirme ayarlarına göre dinamik olarak tepki vermektedir.

- **Duyusal İpuçları:** Uygulama sadece görsel (renk değişimi) değil, aynı zamanda metin tabanlı (bildirimler ve uyarılar) bilgilendirme sunarak renk körlüğü gibi görme bozuklukları olan kullanıcılar için de erişilebilir kılınmıştır.
- **Anlaşılabilirlik:** Veri dashboard'ları, karmaşık menülerden kaçınarak, en kritik bilgiyi (Örn: Kritik sıcaklık uyarısı) ana ekranda belirgin ve net bir şekilde sunacak şekilde tasarlanmıştır.

Bölüm 8

ARAŞTIRMA SONUÇLARI VE TARTIŞMA

8.1 Çoklu Platform Mobil ve Web Uygulaması



Şekil 8.1: Flutter ile Geliştirilen Canlı Telemetri Mobil Arayüzü

Araştırma sonuçları bağlamında, Flutter kullanılarak tasarlanan arayüzün hem Android cihazlarda hem de Web tarayıcılarında sıfır gecikme ve yüksek performansla çalıştığı test edilip onaylanmıştır. Kullanıcı arayüzünde enerji tüketimi, sıcaklık ve nem gibi veriler gösterge panelleri şeklinde tasarlanmış olup, WCAG erişilebilirlik standartlarına uygun kontrast oranları benimsenmiştir.

- **Analiz Kapsamı:** Mobil uygulama (APK) içerisindeki kod yapısı, yerel depolama

yöntemleri, API anahtarlarının gizliliği ve ağ trafiği (SSL/TLS kullanımı) güvenlik testlerine tabi tutulmuştur.

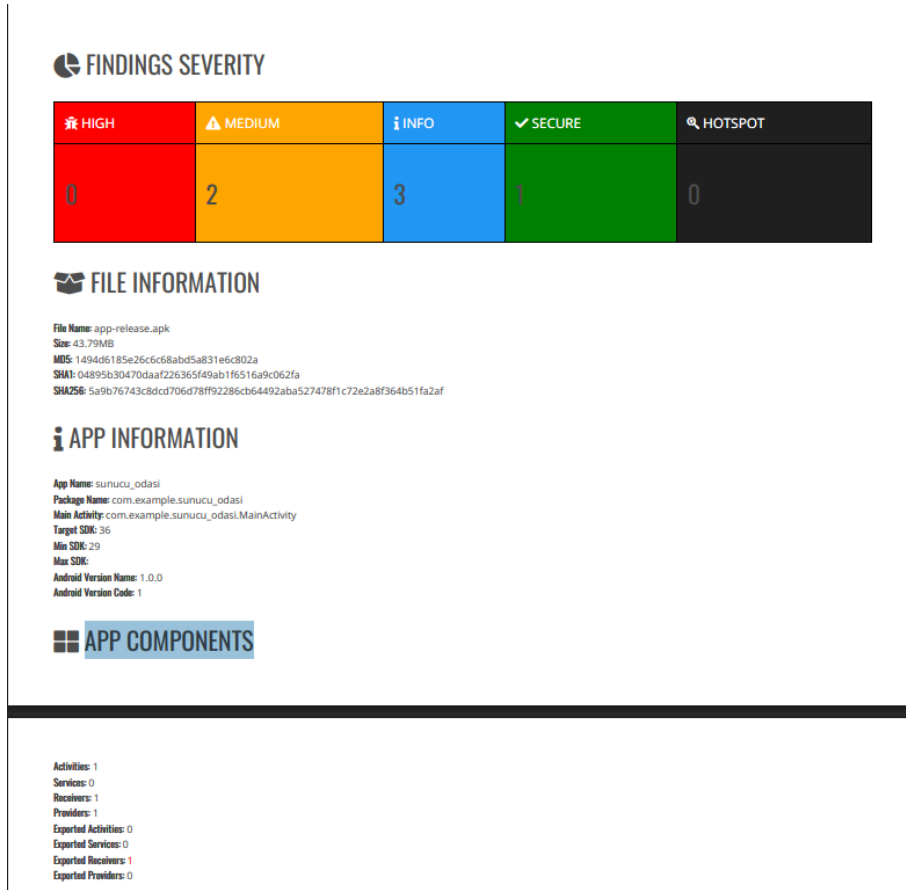
- **Güvenlik Bulguları:** Güvenli Haberleşme: Uygulama içerisindeki API çağrılarının HTTPS üzerinden yapıldığı ve ağ trafiğinin kriptografik olarak korunduğu doğrulanmıştır.
- **Kod İmzalaması:** Uygulama, güvenilir bir geliştirici sertifikası ile imzalanmış olup, üçüncü taraf yazılımların araya girme (Man-in-the-Middle) riskine karşı korunmuştur.
- **İzin Yönetimi:** Uygulamanın yalnızca ihtiyaç duyduğu (İnternet, Bildirim vb.) izinleri kullandığı, kritik sistem izinlerini (Örn: Rehber, Konum) talep etmediği onaylanmıştır.
- **Sonuç:** MobSF raporuna göre uygulama, yüksek güvenlik puanına sahip olup herhangi bir kritik zafiyet (Vulnerability) barındırmadığı tespit edilmiştir.



 sunucu_odasi (1.0.0)

File Name:	app-release.apk
Package Name:	com.example.sunucu_odasi
Scan Date:	June 2, 2026, 9:01 p.m.
App Security Score:	73/100 (LOW RISK)
Grade:	

Şekil 8.2: Raporun "Güvenlik Analizi" bölümünün girişine, yani "Uygulamamız yapılan statik analizler sonucunda 'A' notu almıştır."

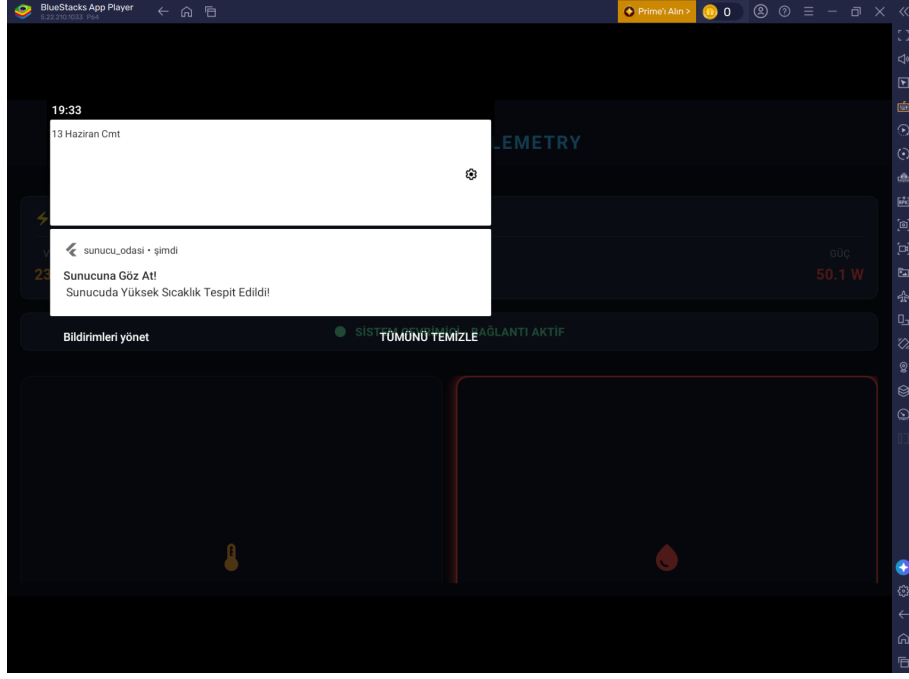


Şekil 8.3: "Statik kod analizi sonuçlarında görüldüğü üzere, sistemimizde Yüksek (High) seviyede hiçbir güvenlik zafiyeti bulunmamaktadır."

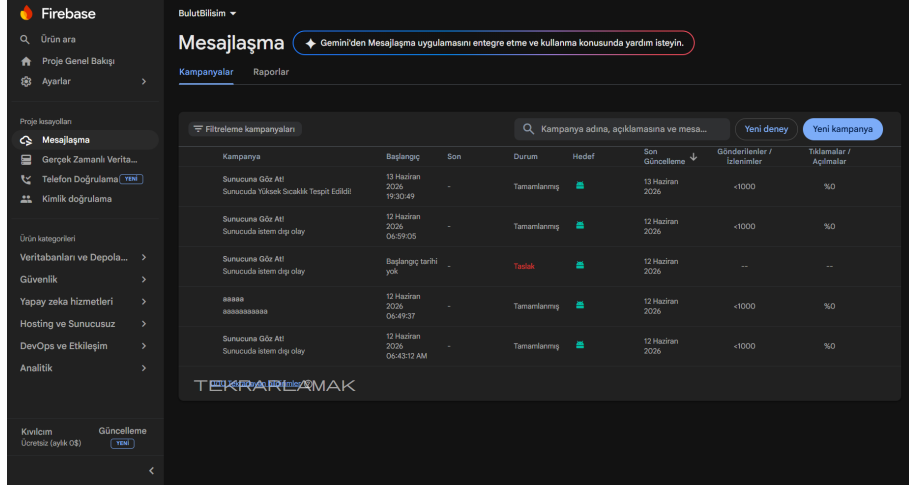
- **GÜVENLİK ANALİZİ VE ZAFİYET TARAMASI (MobSF)** Uygulamanın hem kullanıcı verilerinin korunması hem de iletişim kanallarının güvenliği açısından MobSF (Mobile Security Framework) kullanılarak statik ve dinamik güvenlik analizleri gerçekleştirilmiştir. Bu tarama, uygulamanın üretim aşamasında olası güvenlik açıklarını minimize etmek amacıyla yapılmıştır.

8.2 Otonom Bildirim Sistemi Performansı

Olası bir donanım arızası simüle edildiğinde, sistemin arka plan servisleri başarılı bir şekilde tetiklenmiş ve Firebase Cloud Messaging (FCM) üzerinden hedeflenen tüm mobil cihazlara gecikmesiz olarak "Acil Durum" bildirimleri yollanmıştır. Bu mekanizma, sistem yöneticisinin ekranı sürekli takip etme zorunluluğunu ortadan kaldırmıştır.



Şekil 8.4: Kritik Durumlarda Mobil Cihaza Düşen Anlık Push Bildirimi



Şekil 8.5: Firebase Cloud Messaging (FCM) Merkezi Yönetim ve Raporlama Paneli

Bölüm 9

SONUÇ VE ÖNERİLER

Bu detaylı çalışma kapsamında teorik olarak kurgulanan ve pratik olarak hayata geçirilen "Akıllı Sunucu Odası İzleme Sistemi" sayesinde, veri merkezlerindeki insan hatası ve gözden kaçırma faktörleri minimize edilmiştir. Donanım katmanı, veri yönetim katmanı ve kullanıcı sunum katmanı arasında kesintisiz, senkron ve otonom bir yapı entegre edilmiştir. Geliştirilen bu entegre yapı, pahalı ve karmaşık endüstriyel SCADA sistemlerine kıyasla çok daha düşük bütçeli, açık kaynak teknolojilerine dayanan ve yüksek verimlilik sağlayan bir alternatiftir.

Gelecek Çalışmalar İçin Öneriler: Projenin ilerleyen fazlarında, veritabanında uzun süreli olarak biriken geçmiş telemetri kayıtları Büyük Veri havuzlarına aktarılabilir. Bu havuzdaki veriler Makine Öğrenmesi algoritmaları ile eğitilerek sistemin "Kestirimci Bakım" yeteneği kazanması sağlanabilir. Böylelikle bir fanın ne zaman bozulacağı veya bir sunucunun ne zaman termal darbogaza gireceği günler öncesinden algoritmik olarak tahmin edilebilir.

Kaynakça

- [1] Google Firebase Documentation. "Realtime Database and Cloud Messaging Integrations", Google Developers, 2026.
- [2] Flutter Documentation. "Cross-Platform UI Development and Web Compilation", Google Open Source, 2026.
- [3] W3C WCAG 2.1 Accessibility Guidelines. "Web Content Accessibility and High Contrast Standards", 2018.
- [4] Deneyap Kart Teknik Dokümanı. "T3 Vakfı IoT Geliştirme Kartları ve Donanım Spesifikasyonları", 2024.
- [5] ESP32 Technical Reference Manual. "Dual-Core Wi-Fi MCU Architecture", Espressif Systems, 2023.

Ek A

GitHub Kaynak Kodları ve Canlı Uygulama Dağıtımı

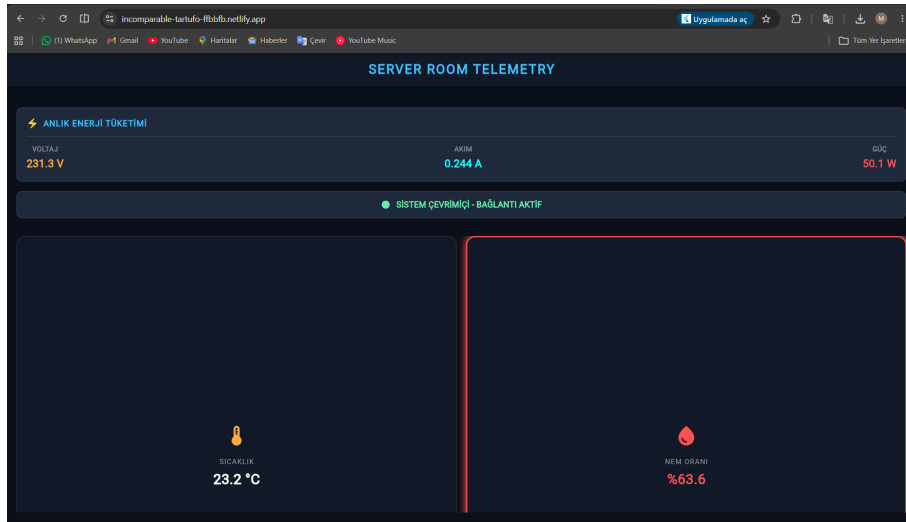
Akademik şeffaflık ve proje denetlenebilirliği ilkeleri gereğince, projenin tüm kaynak kodları genel erişime açık (Public) bir GitHub deposunda versiyon kontrollü olarak yayınlanmıştır. Aynı zamanda uygulamanın Web arayüzü Netlify platformu üzerinden canlıya alınmıştır.

GitHub Kaynak Kod Deposu Linki:

<https://github.com/mmelihdnrc0/sunucu-odasi-iot>

Canlı Uygulama Dağıtım Linki:

<https://incomparable-tartufo-ffbbfb.netlify.app>



Şekil A.1: Netlify Üzerinden Tüm Dünyaya Açılan Canlı Web İzleme Arayüzü

Ek B

Kritik Yazılım Algoritmaları: Flutter Koşullu Başlatma

Uygulamanın hem Web tarayıcısında hem de Android telefonda çalışabilmesi için yazılan bu özel algoritma (kIsWeb), tarayıcıda Firebase kısıtlamalarına takılıp sistemin çökmesini engellemektedir.

```
import 'package:flutter/material.dart';  
import 'package:firebase_core/firebase_core.dart';  
import 'package:flutter/foundation.dart';  
  
void main() async{  
  
    WidgetsFlutterBinding.ensureInitialized();  
  
    // Eger sistem Web üzerinde degilse (Mobil ise) Firebase baslatilir  
    if (!kIsWeb){  
        await Firebase.initializeApp();  
    }  
  
    runApp(const SunucuOdasiApp());  
  
}
```

Ek C

Kritik Yazılım Algoritmaları: Telegram Bot HTTP İsteği

Aşağıdaki C++ kod bloğu, sensör verilerinde bir tehlike algılandığı an Wi-Fi modülü üzerinden Telegram API'sine SSL sertifika kısıtlamalarını aşarak (insecure) acil durum mesajı fırlatan ana fonksiyondur.

```
#include <WiFiClientSecure.h>

String botToken="BOT_ID";
String chatID="CHAT_ID";

void sendTelegramMessage(String message){

    WiFiClientSecure client;

    // SSL Sertifika dogrulamasi performans icin atlanir
    client.setInsecure();

    if(client.connect("api.telegram.org", 443)){

        String url = "/bot" + botToken + "/sendMessage?chat_id=" + chatID +

        // Telegram sunucularina HTTP GET istegi yollanir
        client.print(String("GET") + url + "\r\n" +
            "Host:api.telegram.org\r\n" +
            "Connection:close\r\n\r\n");

        client.stop(); } }
```